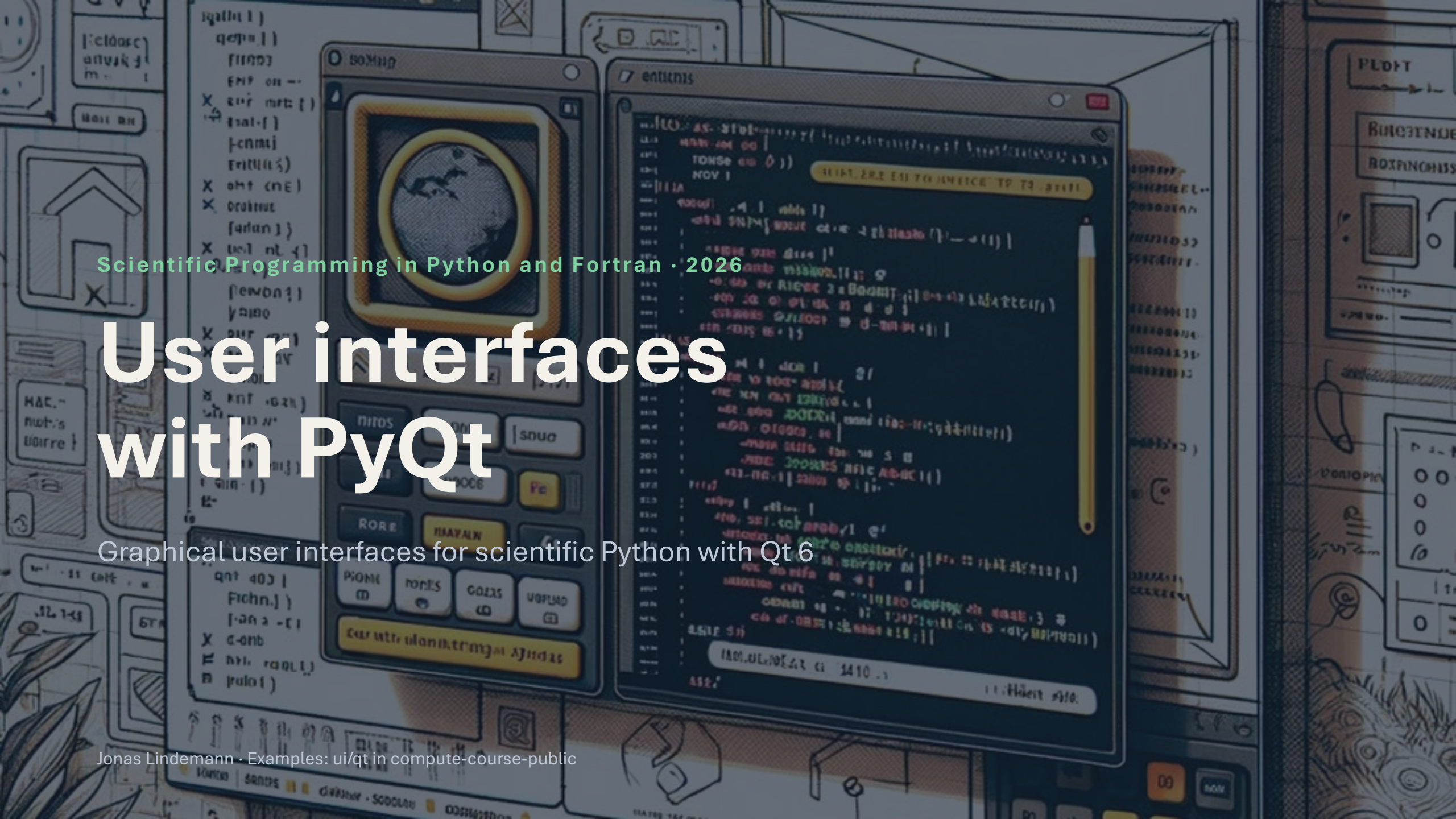


Scientific Programming in Python and Fortran · 2026

User interfaces with PyQt

Graphical user interfaces for scientific Python with Qt 6

Jonas Lindemann · Examples: ui/qt in compute-course-public

Why build a GUI for research code?

Explore parameters

Sliders and live plots turn a script into an instrument. See the result respond while you change the input.

Share with others

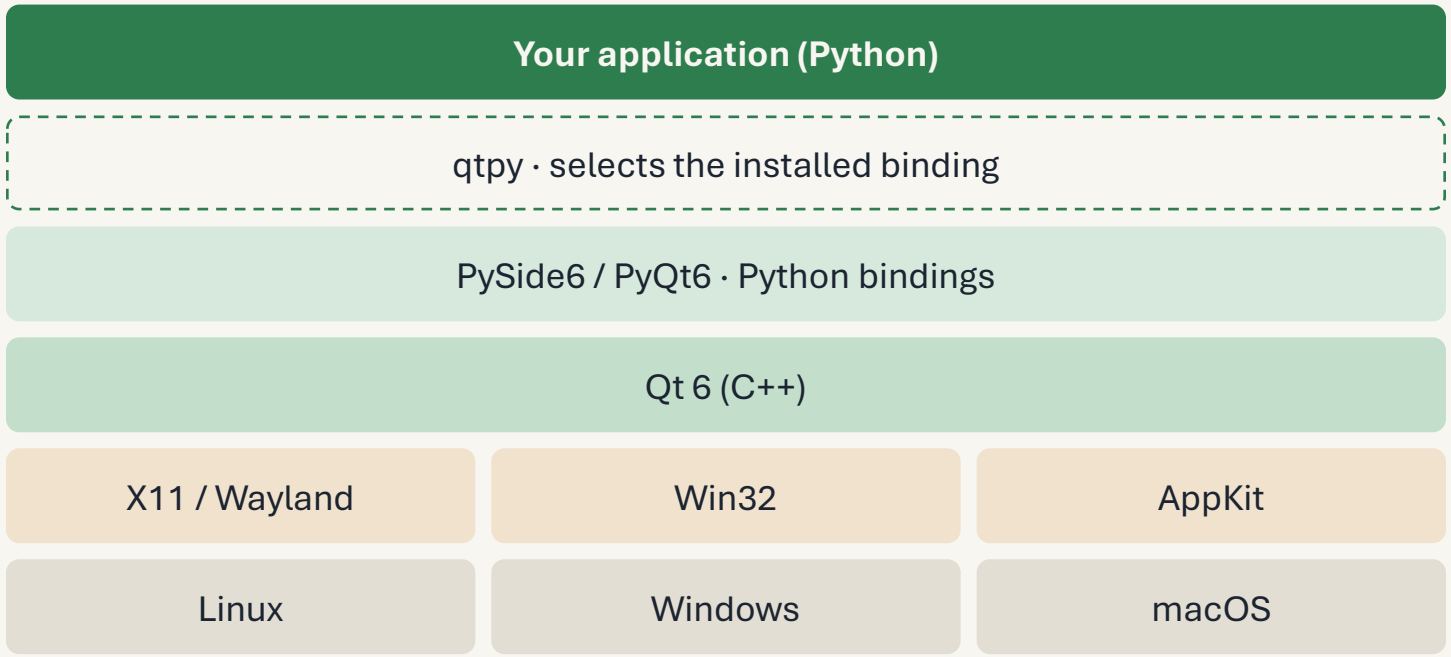
Colleagues, students and partners can run your model without reading or editing the code.

Inspect while running

Follow a long simulation, pause it and look at intermediate results instead of waiting for a log file.

Goal for today: from a ten-line window to applications like the **surface explorer** in cases/surface.

Qt hides the platform differences



Your code only talks to **Qt**.

Qt talks to each platform's **native window system**, so the application looks and behaves like a native one.

qtpy lets the same code run on either Python binding.

What Qt gives you

Widgets

Buttons, inputs, tables, trees, dialogs, menus, docks

Native look

Adapts to each platform, or styled with style sheets

Signals and slots

Loosely coupled event handling between objects

Layouts

Automatic placement and resizing of controls

Model/view

Tables and trees that scale to large data sets

Threads and timers

Background work without freezing the interface

C++ framework, first released in 1995, used in everything from scientific tools to car dashboards.

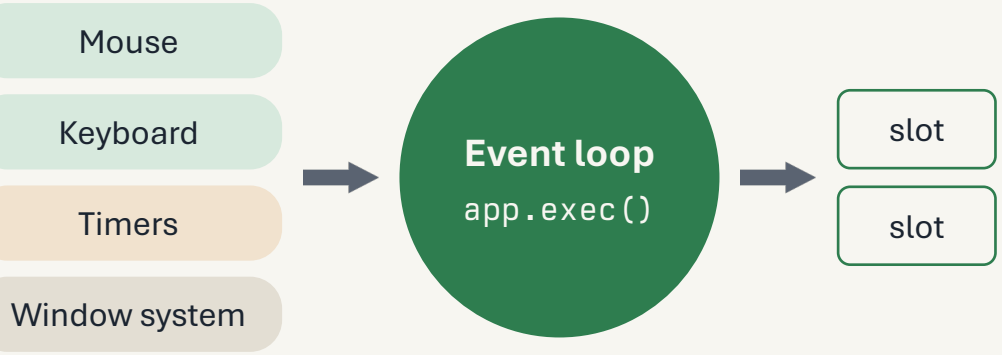
Two Python bindings, one API

	PySide6	PyQt6
Maintained by	The Qt Company (Qt for Python)	Riverbank Computing
License	LGPL v3 or commercial	GPL v3 or commercial
Install	pip install pyside6	pip install pyqt6
Designer files	pyside6-uic, QUiLoader	pyuic6, uic.loadUi

In this course: qtpy

Import from **qtpy** and the same code runs on either binding. Choose explicitly with the environment variable **QT_API=pyside6** when both are installed. The examples are tested with PySide6.

GUI programs are event driven




```
running = True
while running:
    event = wait_for_event()
    if event.type == CLICK:
        call_connected_slots(event)
    elif event.type == PAINT:
        redraw(event.window)
    elif event.type == QUIT:
        running = False
```

our code runs only inside slots. **While a slot runs, nothing else happens**: no repainting, no input. Remember this for part 7.

The smallest Qt program

```
import sys
from qtpy.QtWidgets import QApplication, QWidget

# One QApplication per program, created first
app = QApplication(sys.argv)

# A widget without a parent becomes a window
widget = QWidget()
widget.resize(250, 150)
widget.setWindowTitle("Hello Qt")
widget.show()

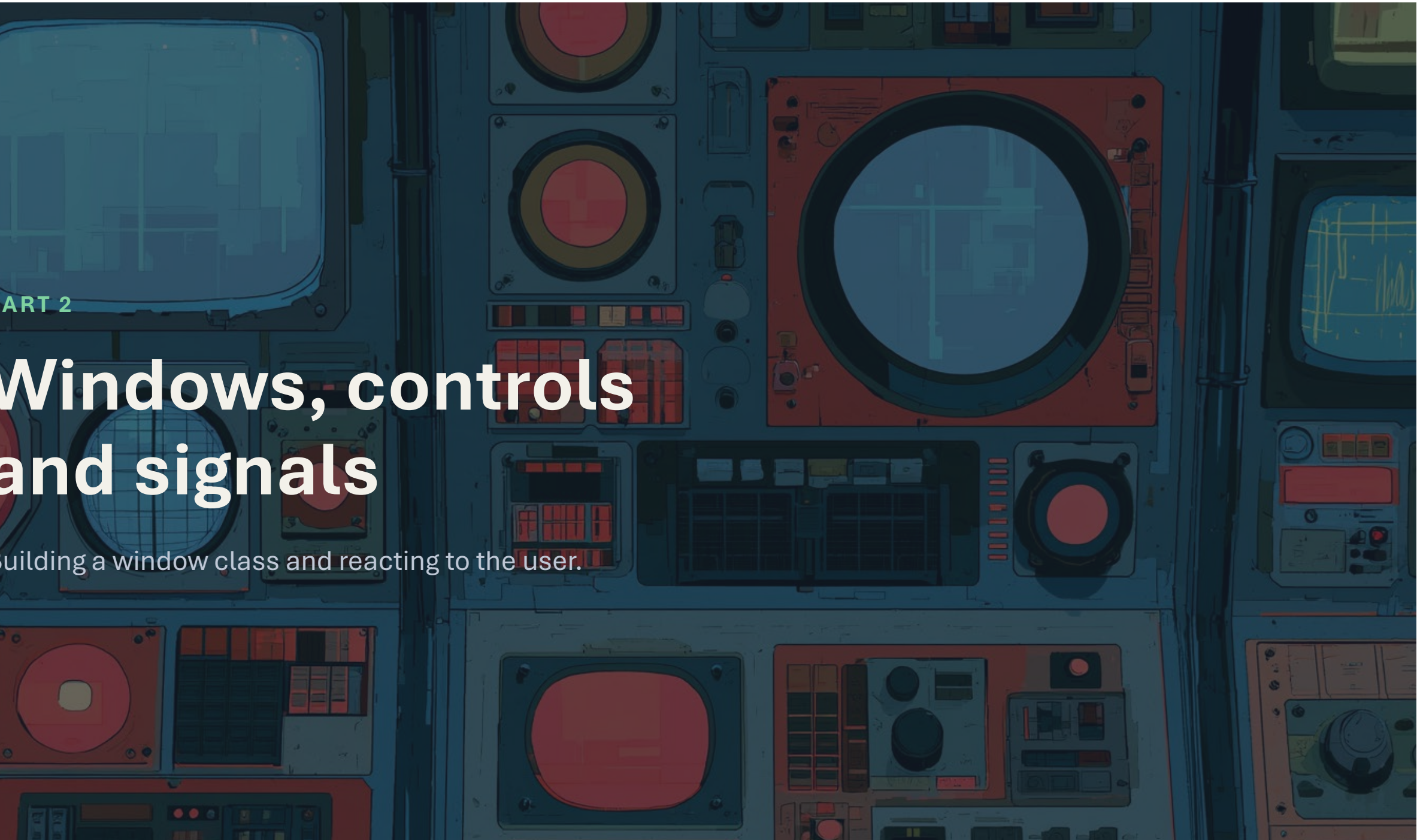
# Blocks until the last window is closed
exit_code = app.exec()
print("Last window closed.")
sys.exit(exit_code)
```

QApplication must exist before any widget is created.

show() only schedules the window. It appears when the event loop runs.

exec() replaces exec_() from Qt 5 examples you may find online.

intro_ex1.py



ART 2

Windows, controls and signals

uilding a window class and reacting to the user.

Every control is a QWidget in a tree

```
MyWindow (QWidget)
├── QLabel
├── QLineEdit
└── QGroupBox
    ├── QRadioButton
    └── QRadioButton
```

QWidget is the base class of all controls.

A widget **without a parent** becomes a window.

A parent **owns** its children: they are shown, hidden and deleted with it.

Adding a widget to a layout sets its parent for you.

A window class

```
import sys
from qtpy.QtWidgets import QApplication, QWidget

class MyWindow(QWidget):
    """Main window class"""

    def __init__(self):
        super().__init__()

        self.setGeometry(300, 300, 600, 600)
        self.setWindowTitle("MyWindow")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MyWindow()
    window.show()
    sys.exit(app.exec())
```

Derive from **QWidget**, or from **QMainWindow** for menus and toolbars.

Call **super().__init__()** first.

Create controls, layouts and connections in **__init__**.

Show the window from the main program.

Adding a control

```
class MyWindow(QWidget):
    def __init__(self):
        super().__init__()

        self.button = QPushButton("Press me", self)
        self.button.setToolTip("I am a button. Please press me")
        self.button.resize(100, 50)
        self.button.move(50, 50)

        self.button.clicked.connect(self.on_button_clicked)

        self.setGeometry(300, 300, 300, 300)

    def on_button_clicked(self):
        print("Hello")
```

Passing **self** as parent puts the button inside the window.

move() and **resize()** place it in pixels: absolute positioning.

> **Fragile.** Breaks when the window is resized, on high-DPI screens and with longer translated text. Part 3 replaces it with layouts.

Signals and slots




```
elf.button.clicked.connect(self.on_button_clicked)
```




```
elf.line_edit.textChanged.connect(self.label.setText)
```

signal is emitted when something happens. A **slot** is any callable. Arguments of the signal are passed on to the slot.

A slot that reads another control

```
class MyWindow(QWidget):
    def __init__(self):
        super().__init__()

        self.line_edit = QLineEdit(self)
        self.line_edit.move(50, 20)
        self.line_edit.setText("Text")

        self.button = QPushButton("Press me", self)
        self.button.move(50, 50)
        self.button.clicked.connect(self.on_button_clicked)

    def on_button_clicked(self):
        """Slot: called by the event loop"""
        QMessageBox.information(self, "Text", self.line_edit.text())
```

Keep controls you need later as attributes: **self.line_edit**.

Connect once, in **__init__**.

Name slots **on_<control>_<signal>** so they are easy to find.

Finding signals in the documentation

Qt for Python

Python signatures and examples

doc.qt.io/qtforpython-6

Qt Widgets (C++)

The most complete reference

doc.qt.io/qt-6/qtwidgets-module.html

eading a C++ signal, for example **QSlider** (listed under its base class QAbstractSlider):

```
// C++ docs:  void valueChanged(int value)
# Python:    slider.valueChanged.connect(self.on_value_changed)  # receives an int
```

heck the **inherited members** too: most signals are defined in a base class.

Defining your own signals

```
class LabeledSlider(QWidget):
    valueChanged = Signal(float)

    def __init__(self, name, minimum, maximum, default, decimals=1):
        super().__init__()
        self._scale = 10 ** decimals
        self.slider = QSlider(Qt.Orientation.Horizontal)
        # ... labels and layout ...
        self.slider.valueChanged.connect(self._on_slider_changed)

    def _on_slider_changed(self, raw_value):
        value = raw_value / self._scale
        self.value_label.setText(f"{value:.{self._decimals}f}")
        self.valueChanged.emit(value)
```

```
slider = LabeledSlider("Extent", 1.0, 15.0, 5.0)
slider.valueChanged.connect(self.update_surface)
```

Declare with **Signal(type)** as a **class attribute**.

emit(value) calls every connected slot.

Users of the widget never see the integer QSlider inside.

qtpy provides Signal for both bindings. PyQt6 itself calls it pyqtSignal.

cases/surface/surface_explorer.py

Passing extra arguments to a slot

```
for i in range(5):
    button = QPushButton(str(i))

    # Wrong: i is looked up when the button is clicked, so every button reports 4
    button.clicked.connect(lambda: self.on_number_clicked(i))

    # Right: a default argument binds i now. clicked sends a bool first, so accept it
    button.clicked.connect(lambda checked=False, i=i: self.on_number_clicked(i))

    # Right: functools.partial binds i now
    button.clicked.connect(partial(self.on_number_clicked, i))
```

he example has one row of buttons per variant. Click **2** in each row and compare: **4**, **2**, **2**.

Properties shared by all controls

```
def on_button1_clicked(self):
    """Toggle visibility of button2"""
    self.button2.setVisible(not self.button2.isVisible())

def on_button2_clicked(self):
    """Toggle enabled state of button3"""
    if self.button3.isEnabled():
        self.button3.setEnabled(False)
        self.button3.setText("Not enabled")
    else:
        self.button3.setEnabled(True)
        self.button3.setText("Enabled")
```

Inherited from **QWidget**:

`setVisible()` · `isVisible()`

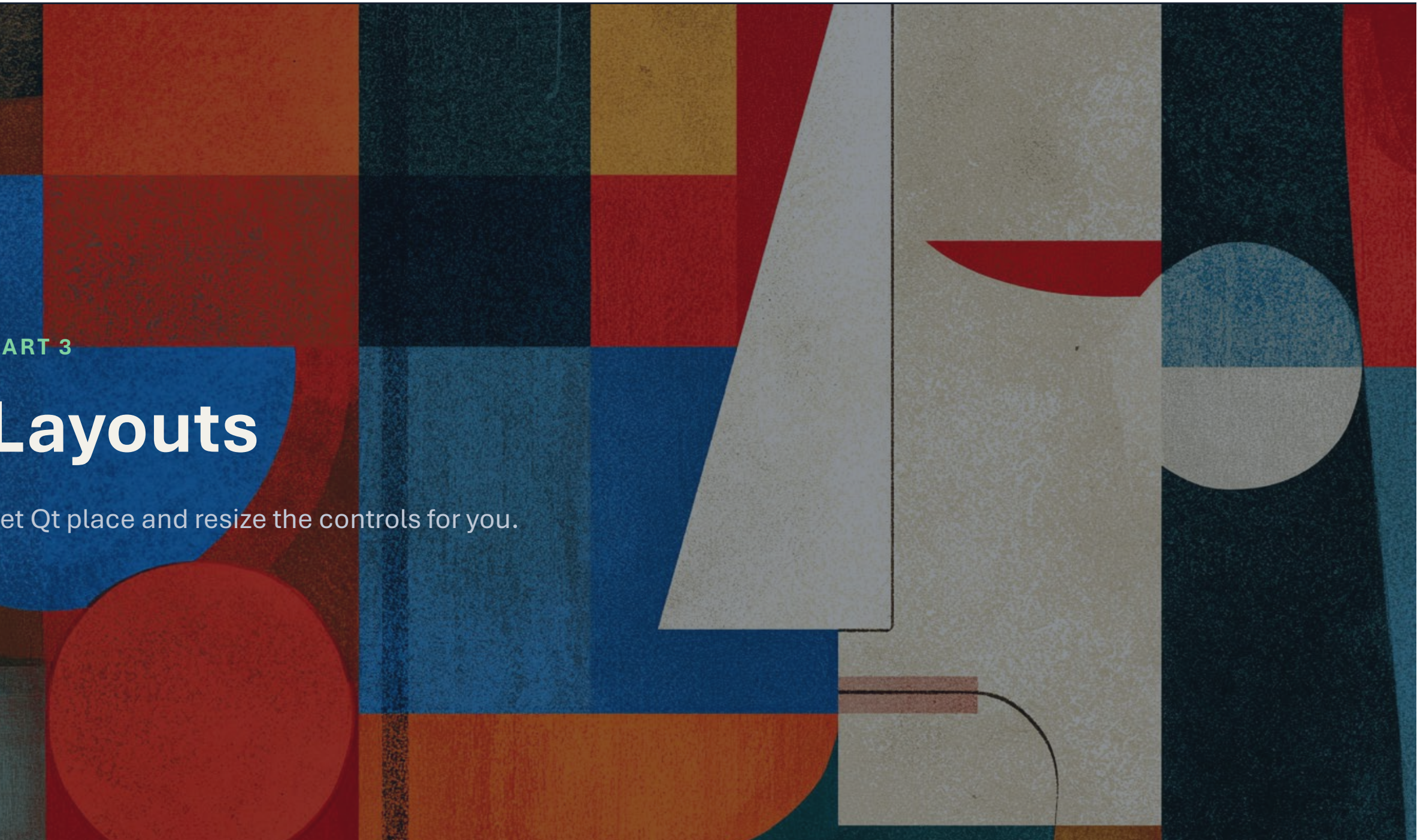
`setEnabled()` · `isEnabled()`

`setToolTip()`

`setFocus()`

`setFont()`

Most text controls add **setText()** and **text()**.



ART 3

Layouts

et Qt place and resize the controls for you.

Why not place controls in pixels?

Resized windows

Controls stay put while the window grows, or get cut off when it shrinks.

Screens and fonts

High-DPI scaling and different system fonts change the size of every control.

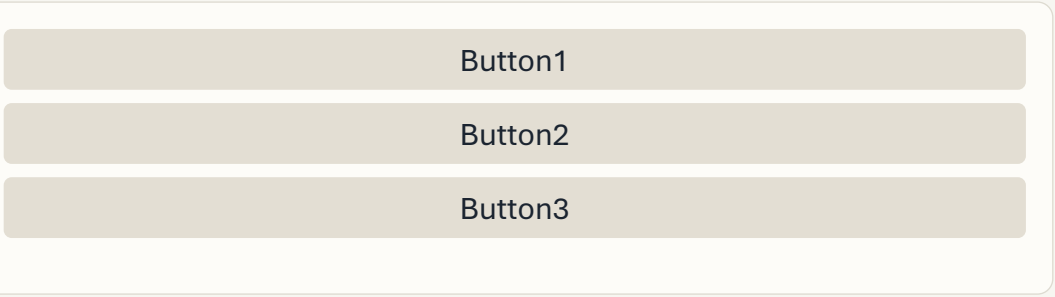
Changing content

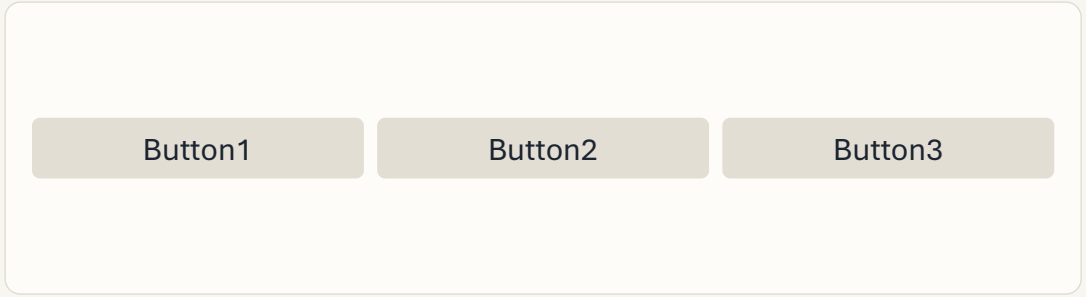
Longer labels, translations or a hidden control leave gaps and overlaps.

ayouts place controls by rules and recompute positions whenever anything changes.

`QVBoxLayout` `QHBoxLayout` `QGridLayout` `QFormLayout`

QVBoxLayout and QHBoxLayout






```
# Passing self sets the window layout
self.vbox = QVBoxLayout(self)
self.vbox.addWidget(self.button1)
self.vbox.addWidget(self.button2)
self.vbox.addWidget(self.button3)
```

```
# Same API, horizontal direction
self.hbox = QHBoxLayout(self)
self.hbox.addWidget(self.button1)
self.hbox.addWidget(self.button2)
self.hbox.addWidget(self.button3)
```

Create controls **without** a parent: adding them to the layout makes them children of the window.

layout_vbox_sizer_1.py · layout_hbox_sizer_1.py

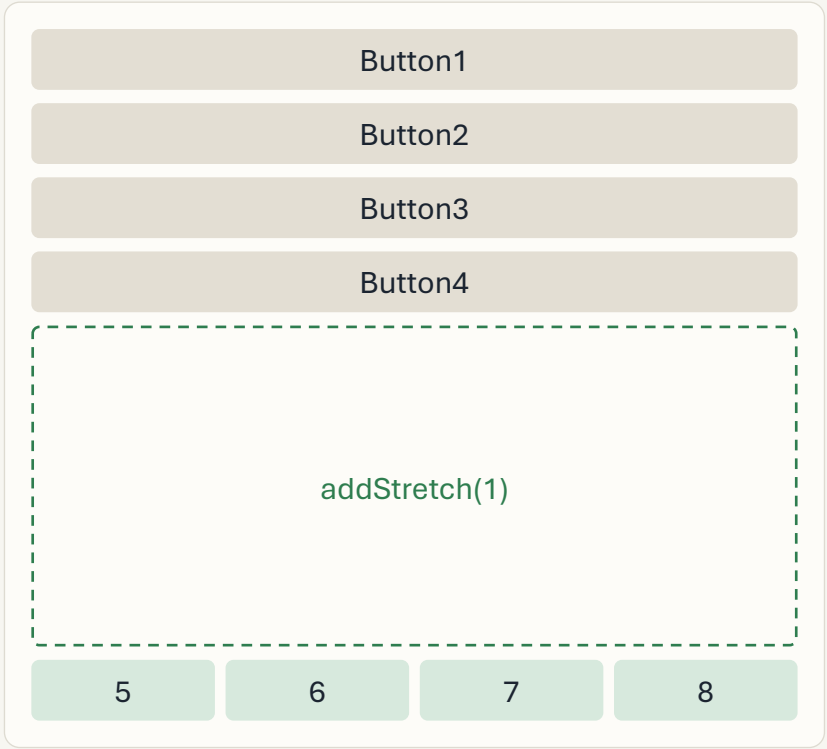
Nesting layouts

```
self.vbox = QVBoxLayout(self)
self.vbox.addWidget(self.button1)
# ... button2 to button4

# Nested layout: no parent
self.hbox = QHBoxLayout()
self.hbox.addWidget(self.button5)
# ... button6 to button8

self.vbox.addStretch(1)
self.vbox.addLayout(self.hbox)
```

QHBoxLayout(self) here would try to give the window a second layout. Qt prints a warning and ignores it.



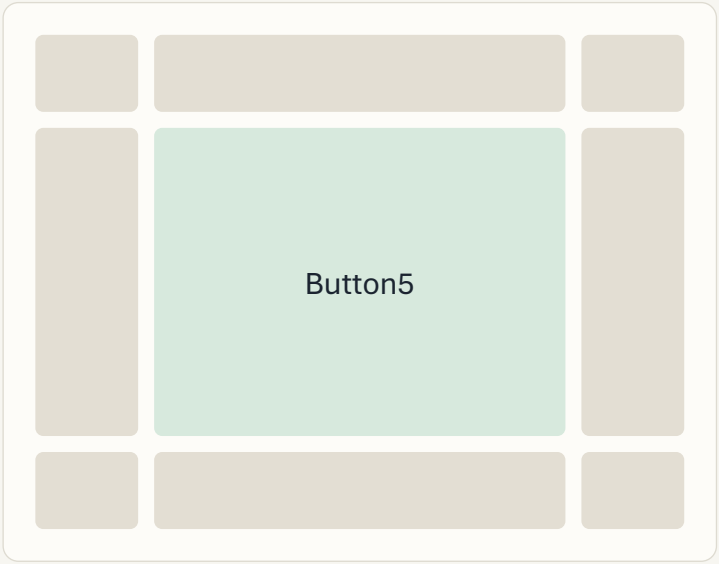
QGridLayout: rows, columns and stretch

```
self.grid = QGridLayout(self)

for row in range(3):
    for col in range(3):
        button = QPushButton(f"Button{row * 3 + col + 1}")
        button.setSizePolicy(QSizePolicy.Policy.Expanding,
                             QSizePolicy.Policy.Expanding)
        self.grid.addWidget(button, row, col)

self.grid.setContentsMargins(20, 40, 20, 40)
self.grid.setHorizontalSpacing(20)

# Middle column gets 4 times the space
self.grid.setColumnStretch(0, 1)
self.grid.setColumnStretch(1, 4)
self.grid.setColumnStretch(2, 1)
# ... the same for rows with setRowStretch
```



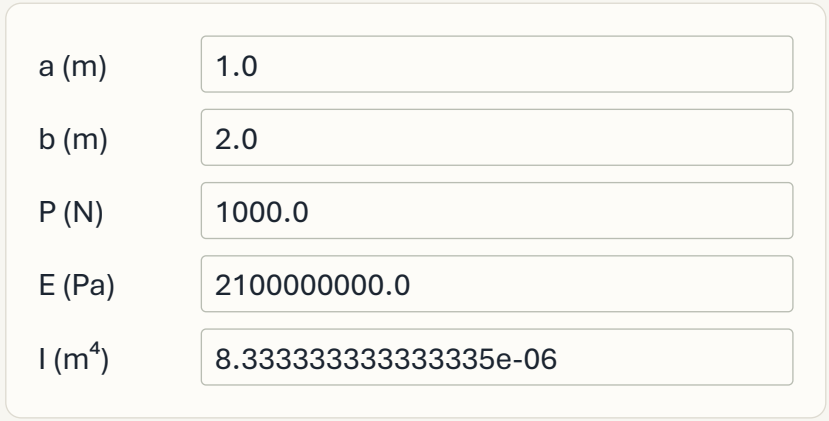
Expanding size policy lets buttons fill their cells.

QFormLayout for parameter input

```
self.a_edit = QLineEdit()
# ... b_edit, P_edit, E_edit, I_edit

form = QFormLayout()
form.addRow("a (m)", self.a_edit)
form.addRow("b (m)", self.b_edit)
form.addRow("P (N)", self.P_edit)
form.addRow("E (Pa)", self.E_edit)
form.addRow("I (m⁴)", self.I_edit)

layout = QVBoxLayout(self)
layout.addLayout(form)
layout.addWidget(self.text_edit)
```



Labels are created for you and aligned the way each platform expects.

Anatomy of a QMainWindow



Fixed areas around **one central widget**.

The central widget can be a plot, a 3D view or a plain QWidget with its own layout.

Docks can be moved, floated and hidden by the user.

cases/surface/surface_explorer.py



ART 4

Controls, actions and dialogs

A compact tour. Each control has its own controls_*.py example.

The common controls at a glance

Control	Key signal	Read / write	Use for
QPushButton	clicked	text() / setText()	Commands
QCheckBox	toggled(bool)	isChecked() / setChecked()	On/off options
QRadioButton	toggled(bool)	isChecked() / setChecked()	One of a few
QComboBox	currentIndexChanged(int)	currentIndex(), currentText()	One of many
QSlider	valueChanged(int)	value() / setValue()	Quick input
QDoubleSpinBox	valueChanged(float)	value() / setValue()	Exact numbers
QLineEdit	editingFinished	text() / setText()	Free text
QListWidget	currentRowChanged(int)	currentRow(), currentItem()	Long lists

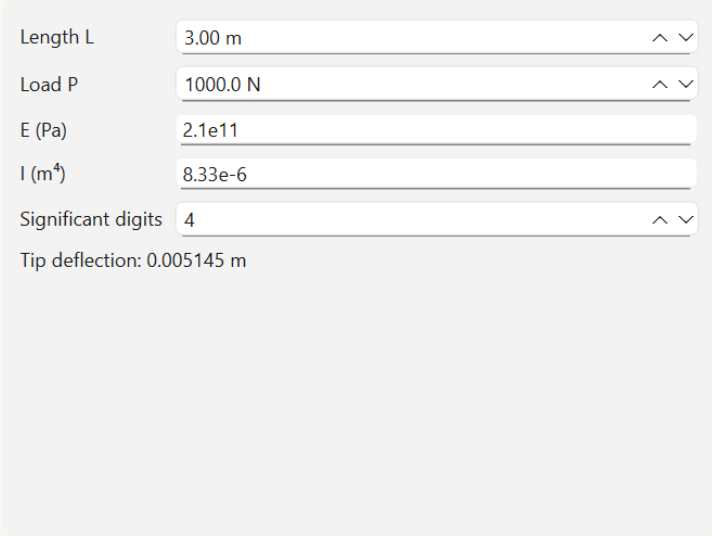
Qt 6: **checkState()** returns an enum that is always truthy. Use **isChecked()** or the **toggled(bool)** signal.

Numeric input without parsing errors

```
self.length_spin = QDoubleSpinBox()
self.length_spin.setRange(0.1, 20.0)
self.length_spin.setDecimals(2)
self.length_spin.setSuffix(" m")

validator = QDoubleValidator(1e-12, 1e15, 6, self)
validator.setNotation(QDoubleValidator.Notation.ScientificNotation)
validator.setLocale(QLocale.c())
self.E_edit = QLineEdit("2.1e11")
self.E_edit.setValidator(validator)

# In the slot: "2.1e" while typing is not acceptable
if not self.E_edit.hasAcceptableInput():
    return
```



Spin boxes can never hold an invalid value. **Validated line edits** suit values over many orders of magnitude.

QLocale.c() keeps the decimal point, also on Swedish systems that expect a comma.

`controls_numeric_input.py`

One QAction, used in menus and toolbars

```
self.open_file_action = QAction(icon("icons8-open.png"), "&Open...", self)
self.open_file_action.setShortcut(QKeySequence.StandardKey.Open)
self.open_file_action.triggered.connect(self.on_open_file_action)

self.file_menu = self.menuBar().addMenu("&File")
self.file_menu.addAction(self.open_file_action)

self.file_toolbar = self.addToolBar("File")
self.file_toolbar.addAction(self.open_file_action)
```

efined once

ext, icon, shortcut and slot live in the action. isable it and every menu entry and button llows.

Platform shortcuts

StandardKey.Open becomes Ctrl+O on Windows and Linux, ⌘O on macOS.

Qt 6 import

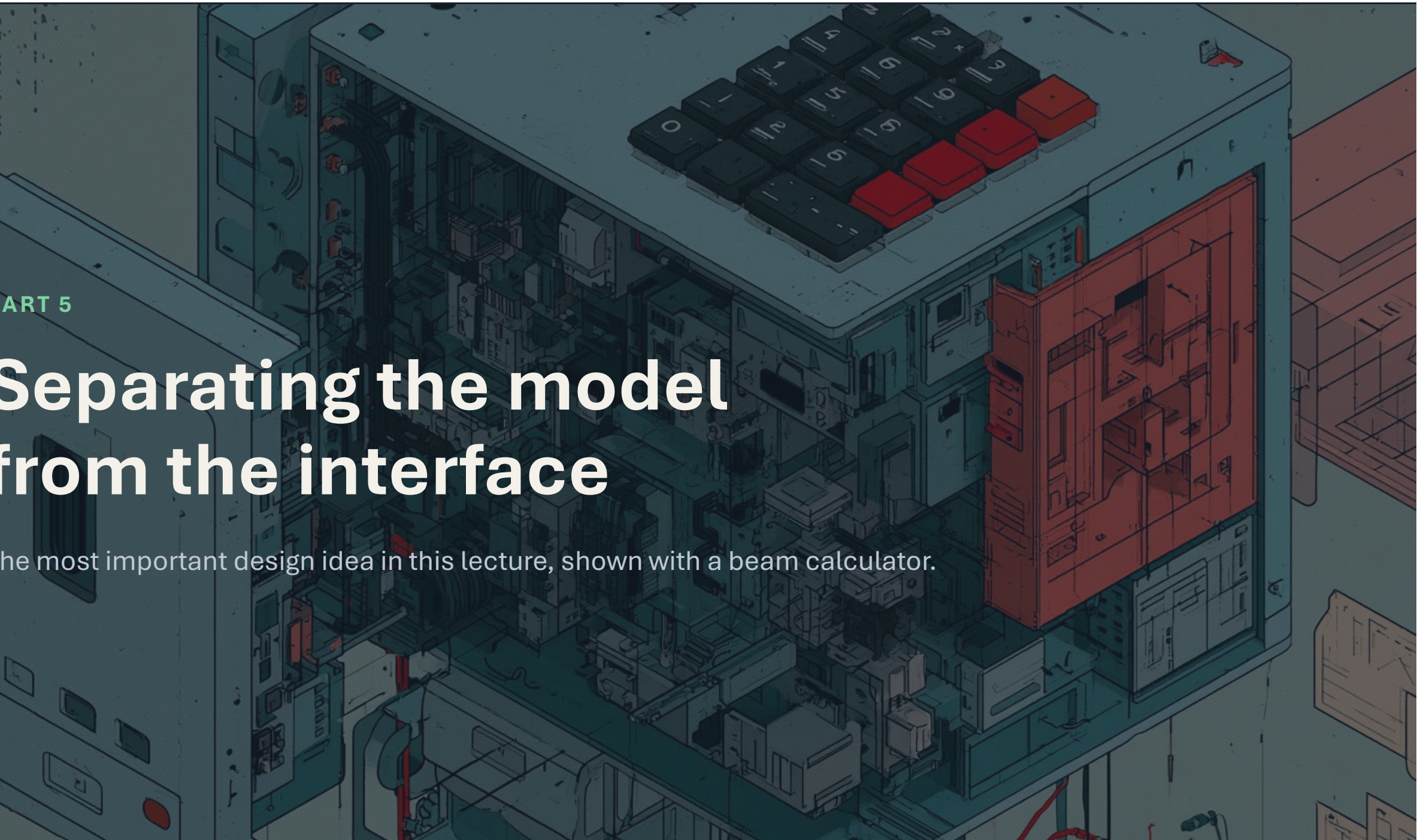
QAction moved to **QtGui**. The **&** marks the Alt-key mnemonic.

Standard dialogs

```
QMessageBox.information(self, "Message", "This is an informative message.")
QMessageBox.warning(self, "Message", "This is a warning message.")
QMessageBox.critical(self, "Message", "This is a critical message.")

reply = QMessageBox.question(
    self, "Message", "Are you sure?",
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
    QMessageBox.StandardButton.No,  # default button
)
if reply == QMessageBox.StandardButton.Yes:
    ...

filename, _ = QFileDialog.getOpenFileName(
    self, "Open file", "", "Python files (*.py);;All files (*)")
if filename:  # empty string if the user cancelled
    ...
```



ART 5

Separating the model from the interface

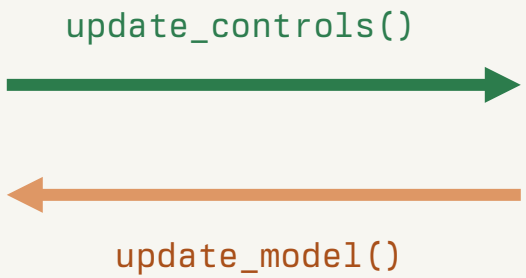
he most important design idea in this lecture, shown with a beam calculator.

Keep computations out of the GUI code

Model

`beam_model_decorators.py`

Data and computations. No Qt imports.



User interface

`beam_ui.py`

Controls, layout and signals. No formulas.

estable without opening a window

Reusable in scripts, notebooks and batch jobs

Replaceable interface: Qt today, a web app tomorrow

Two methods connect model and controls

```
def update_controls(self):
    """Model -> controls"""
    self.a_edit.setText(str(self.beam.a))
    # ... b, P, E and I
    self.update_text_edit()

def update_model(self):
    """Controls -> model"""
    self.beam.a = self.a_edit.text()
    # ... b, P, E and I

def on_editing_finished(self):
    self.update_model()
    self.update_controls()
```

All input fields connect **editingFinished** to the same slot.

Writing back to the controls replaces invalid input with the value the model kept.

Reuse them: call **update_controls()** after loading a file or resetting the model.

The model: a simply supported beam



	$0 \le x < a$	$a \le x \le L$
V(x)	$P \cdot b / L$	$-P \cdot a / L$
M(x)	$-P \cdot b \cdot x / L$	$-P \cdot a \cdot (L - x) / L$
v(x)	$P \cdot b \cdot L/(6EI) \cdot ((1 - b^2/L^2)x - x^3/L^2)$	$P \cdot a/(6EI) \cdot (-a^2 + (2L + a^2/L)x - 3x^2 + x^3/L)$

$L = a + b$

The model class: properties

```
class BeamSimplySupported:
    def __init__(self):
        self.__a = 1.0
        self.__b = 2.0
        self.__I = 0.1 * 0.1**3 / 12.0
        # ... P and E

    @property
    def a(self):
        return self.__a

    @a.setter
    def a(self, v):
        self.__a = self.to_float(v, self.__a)

    @property
    def L(self):
        return self.__a + self.__b
```

to_float() keeps the old value when the input is not a number.

L is derived from a and b, so it is a read-only property.

> Do not store derived values. An earlier version saved L in __init__ and gave wrong results after a or b changed.

The model works without any GUI

```
beam = BeamSimplySupported()

print(f"{'x (m)':>10}  {'v (m)':>10}  {'V (N)':>10}  {'M (Nm)':>10}")

for x in beam.x_values():
    print(f"{x:10.5g}  {beam.v(x):10.5g}  {beam.V(x):10.5g}  {beam.M(x):10.5g}")
```

```
     x (m)       v (m)       V (N)      M (Nm)
        0           0      666.67          -0
      0.1   0.0031683      666.67     -66.667
      0.2   0.0062984      666.67     -133.33
      ...
        3           0     -333.33          -0
```

he same class can be used in a notebook, a parameter study or a test, with no window in sight.

python beam_model_decorators.py

The beam calculator window

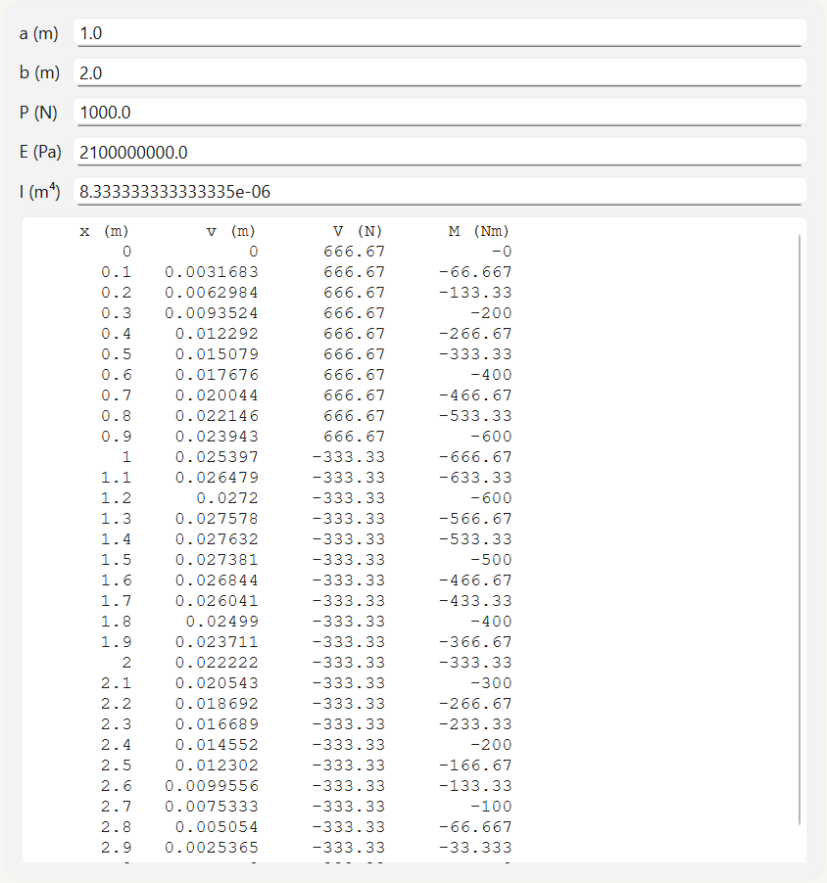
```
class BeamWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.beam = bm.BeamSimplySupported()

        self.a_edit = QLineEdit()
        # ... other fields, form layout, text output

        self.a_edit.editingFinished.connect(
            self.on_editing_finished)
        # ... the same for the other fields

        self.update_controls()
```

Try it: type **abc** in P and press Tab. The field returns to the last valid value.



x (m)	v (m)	V (N)	M (Nm)
0	0	666.67	-0
0.1	0.0031683	666.67	-66.667
0.2	0.0062984	666.67	-133.33
0.3	0.0093524	666.67	-200
0.4	0.012292	666.67	-266.67
0.5	0.015079	666.67	-333.33
0.6	0.017676	666.67	-400
0.7	0.020044	666.67	-466.67
0.8	0.022146	666.67	-533.33
0.9	0.023943	666.67	-600
1	0.025397	-333.33	-666.67
1.1	0.026479	-333.33	-633.33
1.2	0.0272	-333.33	-600
1.3	0.027578	-333.33	-566.67
1.4	0.027632	-333.33	-533.33
1.5	0.027381	-333.33	-500
1.6	0.026844	-333.33	-466.67
1.7	0.026041	-333.33	-433.33
1.8	0.02499	-333.33	-400
1.9	0.023711	-333.33	-366.67
2	0.022222	-333.33	-333.33
2.1	0.020543	-333.33	-300
2.2	0.018692	-333.33	-266.67
2.3	0.016689	-333.33	-233.33
2.4	0.014552	-333.33	-200
2.5	0.012302	-333.33	-166.67
2.6	0.0099556	-333.33	-133.33
2.7	0.0075333	-333.33	-100
2.8	0.005054	-333.33	-66.667
2.9	0.0025365	-333.33	-33.333

beam_ui.py

A separate model is easy to test

```
def test_length_follows_a_and_b():
    beam = make_beam(a=1.0, b=2.0)
    beam.b = 5.0
    assert beam.L == 6.0

def test_midspan_deflection_matches_handbook():
    # Load at midspan: v_max = P L^3 / (48 E I)
    beam = make_beam(a=2.0, b=2.0)
    expected = beam.P * beam.L**3 / (48 * beam.E * beam.I)
    assert isclose(beam.v(beam.L / 2), expected, rel_tol=1e-9)

def test_invalid_input_keeps_old_value():
    beam = make_beam(a=1.0)
    beam.a = "not a number"
    assert beam.a == 1.0
```

Check against **handbook results**, not against the code's own output.

The first test would have caught the stored-L bug.

The UI can be tested too, without showing a window: see **test_beam_ui.py**.

```
pytest test_beam_model.py
```


test_beam_model.py · test_beam_ui.py



ART 6

Showing scientific data

ables, plots, 2D drawing and interactive 3D views.

Two ways to show a table

QTableWidget

`table_item_simple.py`

- Every cell is a QTableWidgetItem
- Data is copied into the widget
- Two copies to keep in sync
- Fine for small tables

QTableView + model

`table_view_model.py`

- The view asks the model for the cells it shows
- Data stays in your array or DataFrame
- One source of truth
- Scales to large data sets

he same idea as part 5: the model owns the data, the view only displays it.

Writing a table model

```
class PandasModel(QAbstractTableModel):
    def __init__(self, data):
        super().__init__()
        self._data = data

    def rowCount(self, parent=QModelIndex()):
        return 0 if parent.isValid() else self._data.shape[0]

    def columnCount(self, parent=QModelIndex()):
        return 0 if parent.isValid() else self._data.shape[1]

    def data(self, index, role=Qt.ItemDataRole.DisplayRole):
        if index.isValid() and role == Qt.ItemDataRole.DisplayRole:
            return str(self._data.iloc[index.row(), index.column()])
        return None

    # headerData() returns the column names
```

data() is called for each visible cell and **role**: text, alignment, colour, tooltip.

Return **None** for roles you do not handle.

Data changed? Wrap the update in **beginResetModel()** and **endResetModel()**.

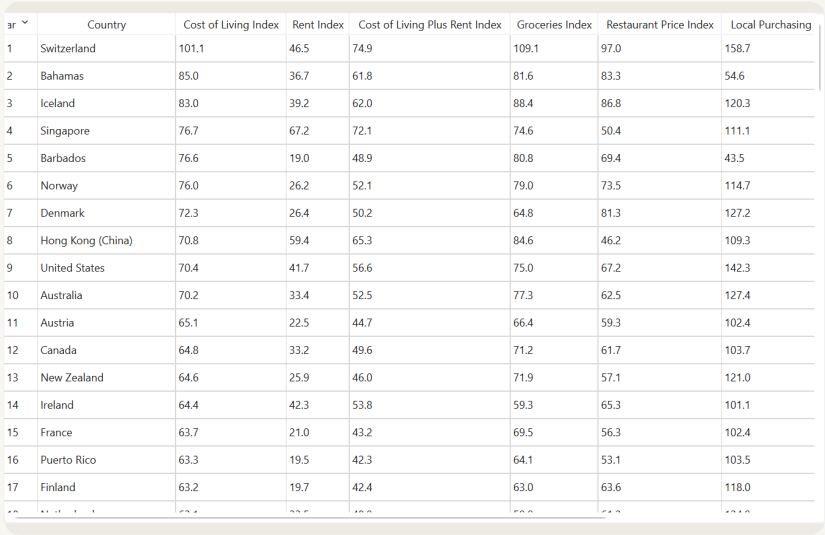
```
view.setModel(model)
```


table_view_model.py

Sorting: the model does the work

```
def sort(self, column, order=Qt.SortOrder.AscendingOrder):
    self.layoutAboutToBeChanged.emit()
    self._data = self._data.sort_values(
        self._data.columns[column],
        ascending=order == Qt.SortOrder.AscendingOrder,
        na_position="last",
    )
    self.layoutChanged.emit()

# In the window
self.table.setSortingEnabled(True)
self.table.sortByColumn(0, Qt.SortOrder.AscendingOrder)
```



ar	Country	Cost of Living Index	Rent Index	Cost of Living Plus Rent Index	Groceries Index	Restaurant Price Index	Local Purchasing
1	Switzerland	101.1	46.5	74.9	109.1	97.0	158.7
2	Bahamas	85.0	36.7	61.8	81.6	83.3	54.6
3	Iceland	83.0	39.2	62.0	88.4	86.8	120.3
4	Singapore	76.7	67.2	72.1	74.6	50.4	111.1
5	Barbados	76.6	19.0	48.9	80.8	69.4	43.5
6	Norway	76.0	26.2	52.1	79.0	73.5	114.7
7	Denmark	72.3	26.4	50.2	64.8	81.3	127.2
8	Hong Kong (China)	70.8	59.4	65.3	84.6	46.2	109.3
9	United States	70.4	41.7	56.6	75.0	67.2	142.3
10	Australia	70.2	33.4	52.5	77.3	62.5	127.4
11	Austria	65.1	22.5	44.7	66.4	59.3	102.4
12	Canada	64.8	33.2	49.6	71.2	61.7	103.7
13	New Zealand	64.6	25.9	46.0	71.9	57.1	121.0
14	Ireland	64.4	42.3	53.8	59.3	65.3	101.1
15	France	63.7	21.0	43.2	69.5	56.3	102.4
16	Puerto Rico	63.3	19.5	42.3	64.1	53.1	103.5
17	Finland	63.2	19.7	42.4	63.0	63.6	118.0

setSortingEnabled(True) sorts at once using the header's default order, which is descending. Call sortByColumn() to choose.

table_view_model_sorted.py · beam_ui_table.py

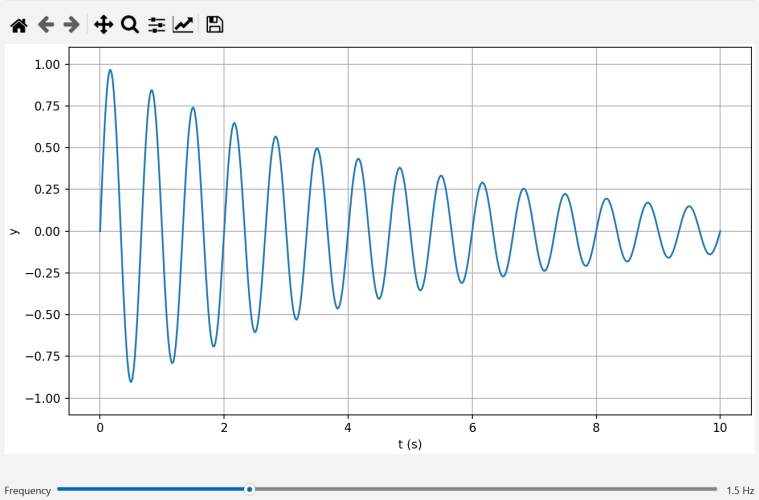
Embedding matplotlib

```
from matplotlib.figure import Figure
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.backends.backend_qtagg import NavigationToolbar2QT

self.figure = Figure(layout="constrained")
self.canvas = FigureCanvasQTAgg(self.figure)
self.toolbar = NavigationToolbar2QT(self.canvas, self)
self.ax = self.figure.add_subplot()
(self.line,) = self.ax.plot(self.x, self.damped_wave(1.0))

layout.addWidget(self.toolbar)
layout.addWidget(self.canvas)

def on_frequency_changed(self, value):
    self.line.set_ydata(self.damped_wave(value / 10.0))
    self.canvas.draw_idle()
```



The canvas is a **QWidget**. Use **Figure**, not pyplot, so no extra windows appear.

Update the line data and call **draw_idle()**: much faster than replotting.

plot_matplotlib.py

pyqtgraph for fast, live plots

```
import pyqtgraph as pg

self.plot_graph = pg.PlotWidget()
self.plot_graph.setLabel("bottom", "Time (min)")
self.plot_graph.setLabel("left", "Temperature (°C)")
layout.addWidget(self.plot_graph)

self.plot_graph.plot(minutes, temperature, pen=pg.mkPen("r", width=2))
```

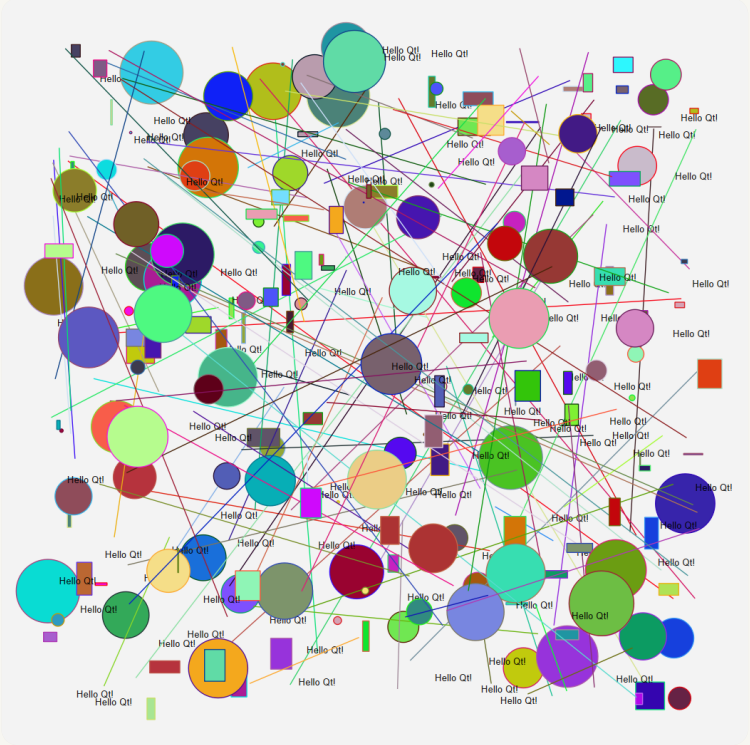
	matplotlib	**pyqtgraph**
Best for	Publication-quality figures	Interactive and live data
Update speed	Moderate	Fast
Zoom and pan	Through the toolbar	With the mouse, built in

Your own 2D drawing: QGraphicsView

```
class DrawingEnv:
    def __init__(self):
        self.scene = QGraphicsScene()
        # ... pens, brushes and font

    def circle(self, x, y, r):
        self.scene.addEllipse(x - r, y - r, 2 * r, 2 * r,
                              self.current_pen, self.current_brush)

# In the window
self.draw = DrawingEnv()
self.graphics_view = QGraphicsView()
self.graphics_view.setScene(self.draw.scene)
self.graphics_view.setRenderHint(QPainter.RenderHint.Antialiasing)
```



scene holds items, a **view** displays it. Items can be selected, moved and zoomed without redrawing everything.

nteractive 3D with PyVista

```
from pyvistaqt import QtInteractor

self.plotter = QtInteractor(self)        # a QWidget with a VTK render window
self.setCentralWidget(self.plotter)

self.mesh = pv.StructuredGrid(X, Y, Z)
self.mesh["Height"] = Z.ravel(order="F")
self.actor = self.plotter.add_mesh(self.mesh, scalars="Height", cmap="viridis")
```

Surface explorer

Parametric surfaces, live parameter sliders, colour maps

`cases/surface`

Volume explorer

Scalar fields with two movable cutting planes

`cases/volume`

Particle explorer

A running simulation animated with a QTimer

`cases/particle`

`pip install pyvista pyvistaqt`



ART 7

Long computations

Keeping the interface responsive while the model runs.

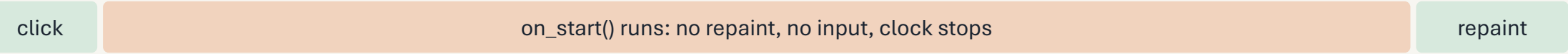
The problem: a slot that takes seconds

```
def on_start(self):
    self.result_label.setText("Computing...")  # never shown

    pi = estimate_pi(N_SAMPLES, CHUNK_SIZE, np.random.default_rng())  # about 4 s

    self.result_label.setText(f"pi ≈ {pi:.6f}")
```

What the event loop does



The window cannot be moved or closed.

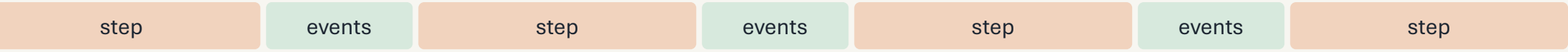
The OS may report **Not responding**.

Users assume the program crashed.

Option 1: small steps driven by a QTimer

```
self.compute_timer = QTimer(self)
self.compute_timer.setInterval(0)  # 0 ms: run whenever the event loop is idle
self.compute_timer.timeout.connect(self.step)

def step(self):
    # Process one chunk. Must return within a few milliseconds.
    x, y = self.rng.random((2, CHUNK_SIZE))
    self.inside += np.count_nonzero(x * x + y * y <= 1.0)
    self.chunk += 1
    self.progress_bar.setValue(self.chunk)
    if self.chunk == self.n_chunks:
        self.compute_timer.stop()
```



+ No threads. Widgets can be updated directly.

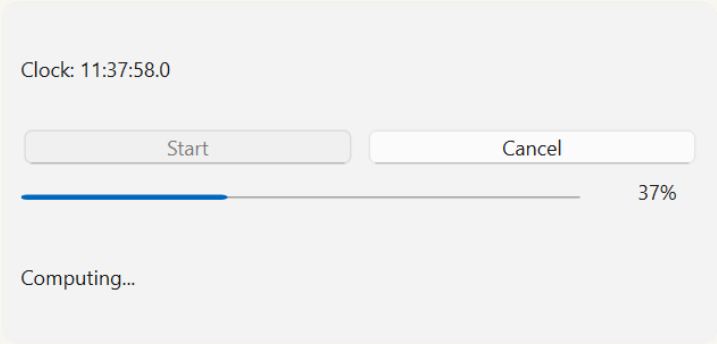
– The computation must be split into short steps.

Option 2: a worker object in a QThread

```
class PiWorker(QObject):
    progress = Signal(int)
    finished = Signal(float)
    cancelled = Signal()

    def run(self):
        # Runs in the worker thread
        for chunk in range(n_chunks):
            if QThread.currentThread().isInterruptionRequested():
                self.cancelled.emit()
                return
            x, y = rng.random((2, self.chunk_size))
            inside += np.count_nonzero(x * x + y * y <= 1.0)
            self.progress.emit(100 * (chunk + 1) // n_chunks)

        self.finished.emit(4.0 * inside / self.n_samples)
```



A plain **QObject**: it owns no widgets.

It reports back **only through signals**.
It checks for cancellation between chunks.

threads_worker.py

Starting the worker

```
self.worker_thread = QThread()
self.worker = PiWorker(N_SAMPLES, CHUNK_SIZE)
self.worker.moveToThread(self.worker_thread)

self.worker_thread.started.connect(self.worker.run)

# Worker -> GUI. These slots run in the GUI thread.
self.worker.progress.connect(self.progress_bar.setValue)
self.worker.finished.connect(self.on_finished)

# Stop the thread when the work is done
self.worker.finished.connect(self.worker_thread.quit)
self.worker_thread.finished.connect(self.on_thread_finished)

self.worker_thread.start()
```

moveToThread(): the worker's slots now run in the new thread.

Signals between threads are **queued** and delivered as events in the receiver's thread.

Keep references in **self**, or Python may delete the thread while it runs.

threads_worker.py

Never touch a widget from a worker thread.

Send data with signals. Qt delivers them to the GUI thread.



Cancelling and cleaning up

```
def on_cancel(self):
    self.worker_thread.requestInterruption()

def on_thread_finished(self):
    self.worker.deleteLater()
    self.worker_thread.deleteLater()
    self.worker = None
    self.worker_thread = None

def closeEvent(self, event):
    # Stop the thread before the window is destroyed
    if self.worker_thread is not None:
        self.worker_thread.requestInterruption()
        self.worker_thread.quit()
        self.worker_thread.wait()
    super().closeEvent(event)
```

Cancelling is **cooperative**: the worker checks the flag between chunks.

Closing the window while a QThread runs aborts the program, so **wait()** first.

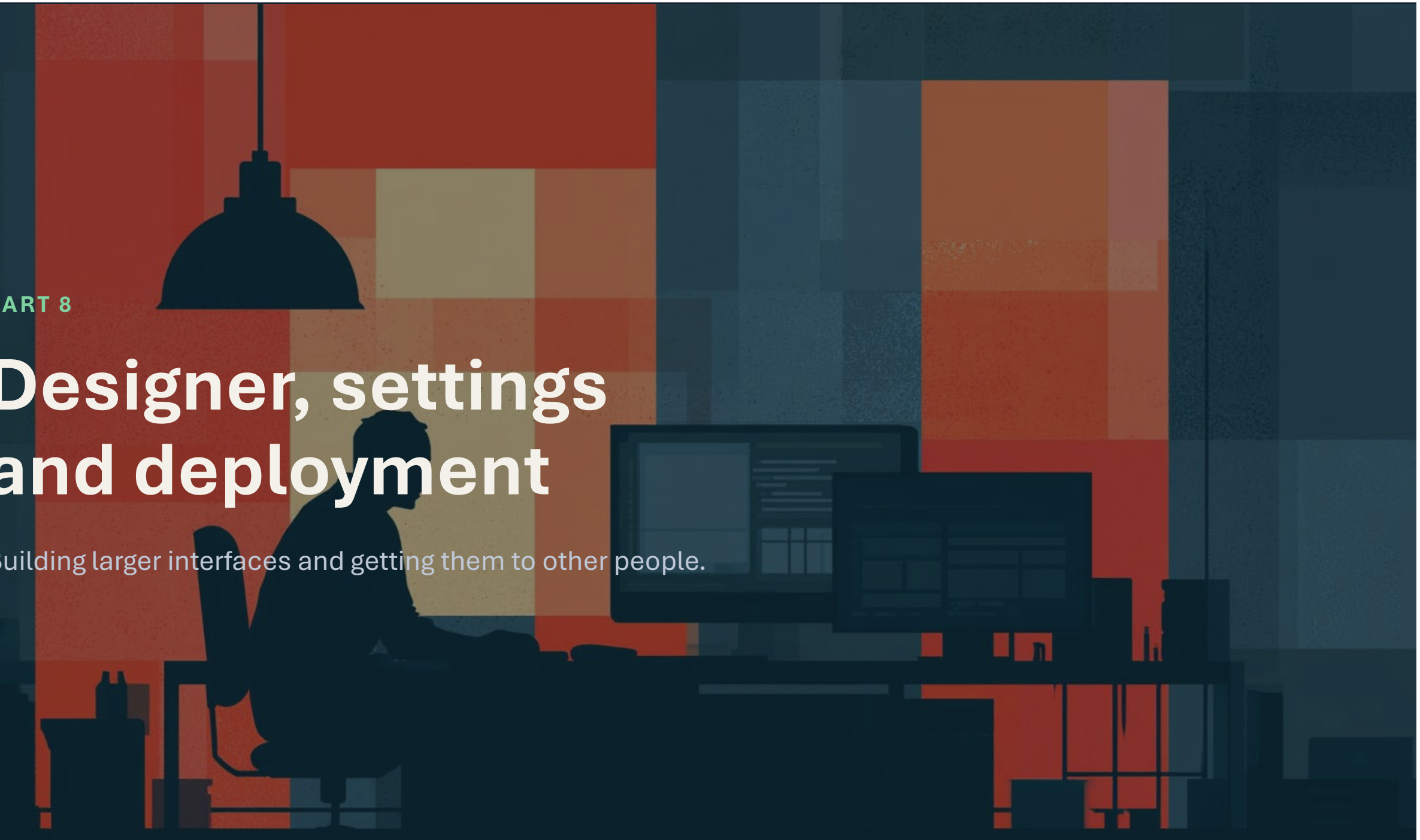
deleteLater() frees Qt objects once pending events are handled.

Choosing an approach

Approach	Use when
QTimer steps	Short repeated steps, such as time stepping
QThread worker	One long task in NumPy, compiled code or I/O
ProcessPoolExecutor	CPU-heavy pure Python, or many independent cases
QProcess	Running an external solver executable

Why not always threads? The GIL.

In standard CPython only one thread runs Python code at a time. NumPy releases this lock inside most array operations, so the π example really runs in parallel with the GUI. A pure-Python loop in a thread keeps the window alive but gains no speed.



ART 8

Designer, settings and deployment

uilding larger interfaces and getting them to other people.

Qt Designer: draw the interface

Drag, drop, lay out

Place widgets, then apply layouts from the toolbar. Preview with Ctrl+R.

Saved as a .ui file

Plain XML describing widgets, layouts and properties. No Python code.

Name every widget

The **objectName** becomes the attribute in Python: self.push_button.

Promote widgets

Place a plain QWidget and promote it to your own class, such as a plot canvas.

tart it with **pyside6-designer**, installed together with PySide6.

Loading a .ui file

```
from pathlib import Path
from qtpy.QtWidgets import QApplication, QWidget
from qtpy import uic

HERE = Path(__file__).resolve().parent

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()

        # Relative to this script, not the working directory
        uic.loadUi(str(HERE / "form.ui"), self)

        self.push_button.setText("Press me!")
```

Widgets become **attributes of self**, named by their objectName.

Connect signals **after** loadUi().

> Import uic from **qtpy**, not from PyQt5 or PyQt6, or the code is tied to one binding.

Load at runtime or compile to Python?

	Load at runtime	Compile to Python
Command	uic.loadUi("form.ui", self)	pyside6-uic form.ui -o ui_form.py
After a change	Save in Designer, run again	Recompile, then run
Editor support	No completion of widget names	Full completion
Distribute	The .ui file with the code	Python files only

```
# Using the compiled file
from ui_form import Ui_Form

self.ui = Ui_Form()
self.ui.setupUi(self)
self.ui.push_button.setText("Press me!")
```

Remembering settings with QSettings

```
self.settings = QSettings("ComputeCourse", "QtExamples")

# Restore in __init__
geometry = self.settings.value("window/geometry")
if geometry is not None:
    self.restoreGeometry(geometry)
self.length_spin.setValue(self.settings.value("params/length", 3.0, type=float))

# Save when the window closes
def closeEvent(self, event):
    self.settings.setValue("window/geometry", self.saveGeometry())
    self.settings.setValue("params/length", self.length_spin.value())
    super().closeEvent(event)
```

tored natively: the **registry** on Windows, a **.plist** on macOS, an **.ini** **type=** converts values that may come back as strings.
le on Linux.

Sharing the application




```
pyinstaller --noconfirm --add-data "beam_app.ui;." --add-data "beam_results.ui;." beam_app.py

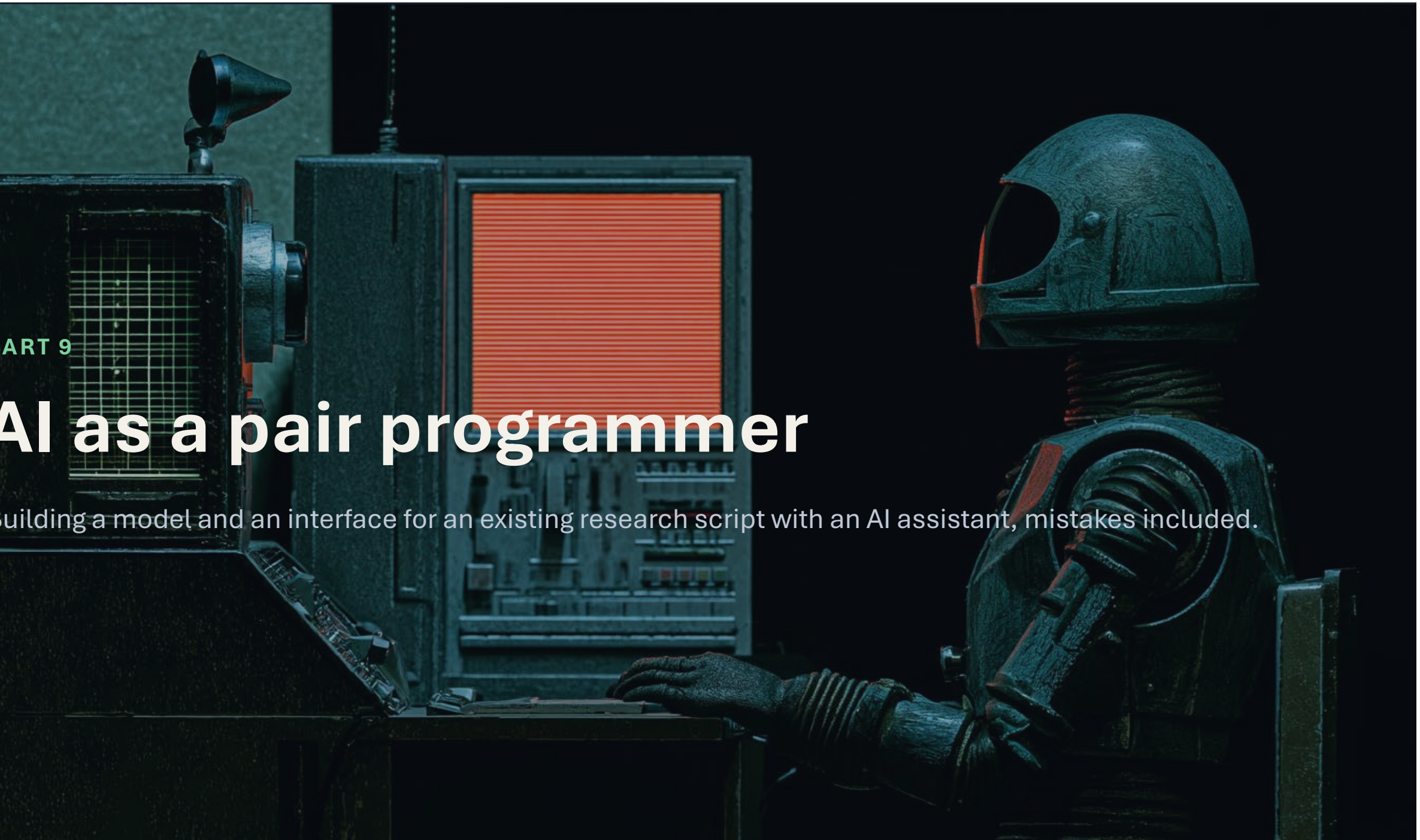
# Find bundled files both when frozen and when run as a script
def resource_path(relative_path):
    base_path = getattr(sys, "_MEIPASS", os.path.dirname(os.path.abspath(__file__)))
    return os.path.join(base_path, relative_path)
```

uild on each target platform. There is no ross-compiling.

.ui files and images are **data**: add them and load them with resource_path().

Alternative: **pyside6-deploy**, based on Nuitka.

cases/beam_app_v2: build_binary.cmd · beam_app.spec · install/



ART 9

AI as a pair programmer

uilding a model and an interface for an existing research script with an AI assistant, mistakes included.

What AI assistants are good and bad at

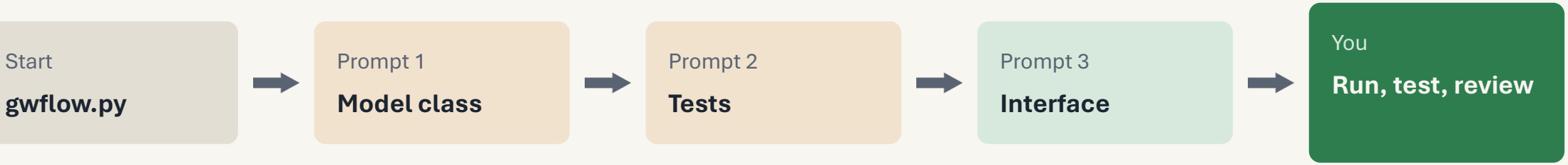
Good at

- Layouts, forms and signal wiring
- Turning a script into a class
- Writing tests once told what to check
- Explaining unfamiliar APIs and errors

Weak at

- Physics, units and valid parameter ranges
- Exact library APIs, especially less common ones
- Questioning the code it was given
- Knowing whether the answer is right

Small steps, not "make me an app"



Each prompt is one well-defined task. The **model/interface split** from part 5 is what makes this possible.

The interface prompt is essentially the **checklist** from this course: qtpy, layouts, update methods, worker thread.

```
ui/qt/ai/prompts.md       # the three prompts
ui/qt/ai/generated/       # the code exactly as first generated
ui/qt/ai/*.py, review.md  # corrected code and what was changed
```

The starting point: a typical research script

```
import calfem.vis as cfv

w = 100.0
h = 10.0
t = 5.0
d = h/2.0
# ... geometry, mesh, assembly ...

a,r = cfc.solveq(K,f,bc,bcVal)
# ... element flows ...

cfv.figure()
cfv.drawElementValues(maxFlow, coords, edof, dofsPerNode, elType,
                      None, doDrawMesh=False, title="Max flows")
# ... three more figures ...
```

2D groundwater flow with CALFEM: a soil region with a notch, head 10 m on the left and 0 m on the right.

Parameters hard-coded at the top. Mesh, solve and plot in one long script.

> It no longer runs: calfem.vis needs the visvis package, which is not installed.

general/lecture4_notebook/gwflow.py

Prompt 1: extract a model

PROMPT

Refactor it into a module gwflow_model.py with a class GroundwaterModel. Parameters as attributes, with the script's values as defaults. solve() creates the geometry and mesh, assembles, solves and stores the results. No plotting and no Qt imports.

```
class GroundwaterModel:
    def __init__(self):
        self.w = 100.0   # Width of the region
        # ... h, t, d, kx, ky, heads

    def solve(self):
        self.geometry = self.create_geometry()
        coords, edof, ... = self.create_mesh(...)
        ep = [1.0, 1]
        # ... assemble, solve, flows
        self.a = a
        self.flow = np.array(flow)
```

Reproduced the original exactly: 1307 nodes, 1189 elements, max flow 0.9798.

ai/generated/gwflow_model.py

Prompt 2: test the physics

PROMPT

Write pytest tests. Test the physics, not only that the code runs: prescribed heads are enforced, heads lie between the boundary values, inflow equals outflow, the solution is antisymmetric when the notch is centred, doubling the permeability doubles the flow.

```
test_prescribed_heads_are_enforced    PASSED
test_heads_between_boundary_values    PASSED
test_inflow_equals_outflow            PASSED
test_antisymmetric_about_centre_line  FAILED
test_double_permeability_doubles_flow PASSED

Obtained: 9.99998531
Expected: 10.0 ± 1.0e-06
```

Right idea, wrong tolerance: the Gmsh mesh is not symmetric, so the discrete solution is only **approximately** antisymmetric.

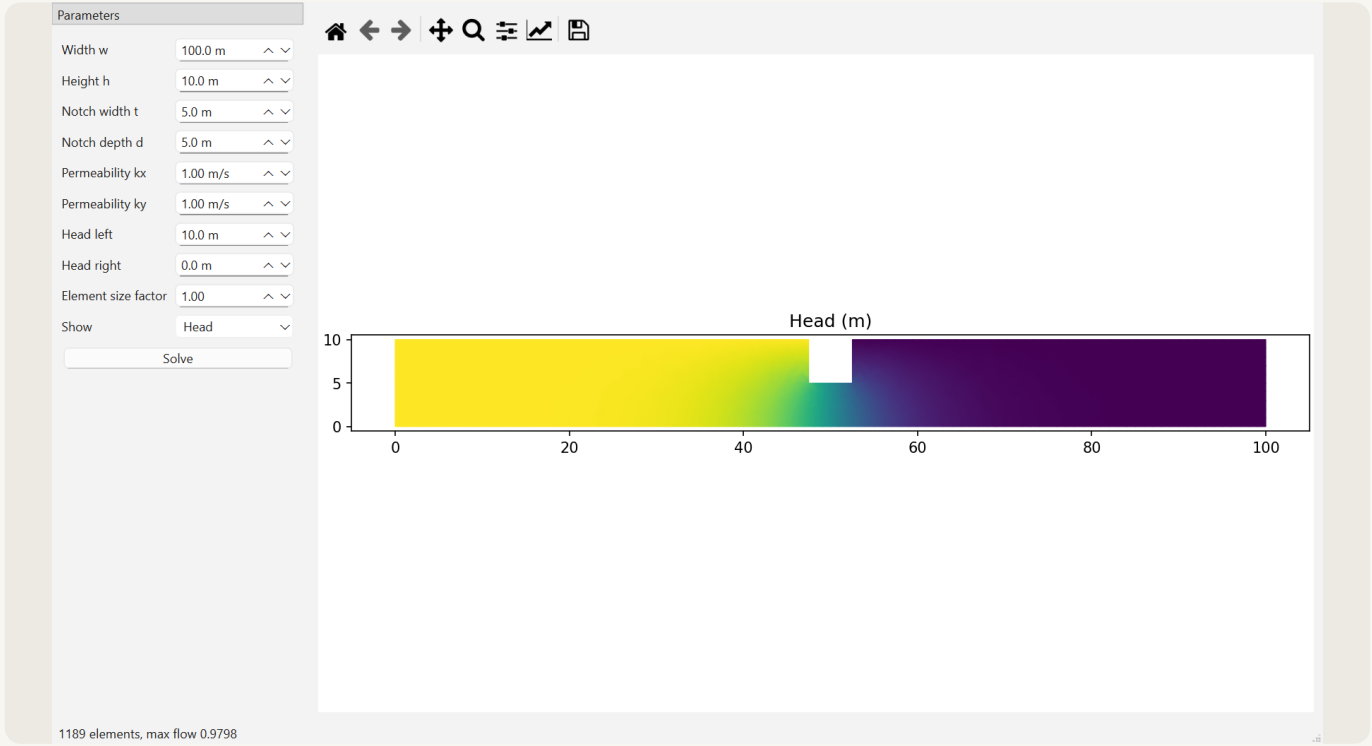
But the error did **not** shrink when the mesh was refined. Following that up found a real problem.

ai/generated/test_gwflow_model.py

Prompt 3: the interface

PROMPT

Import Qt through qtpy, Qt 6 API. Parameters in a dock: QFormLayout of QDoubleSpinBoxes with sensible ranges and units. update_controls() and update_model(). Solve in a worker object in a QThread, inputs disabled while busy. Head or flow, chosen in a combo box, drawn with calfem.vis_mpl in an embedded FigureCanvasQTAgg.

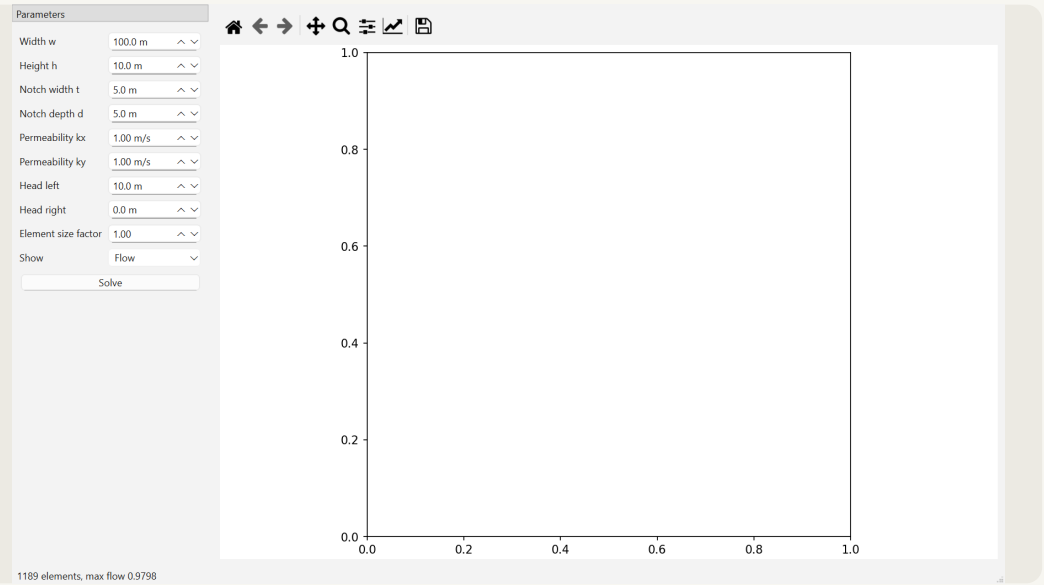


First run: solves in a worker thread with inputs disabled, and the head plot is correct.

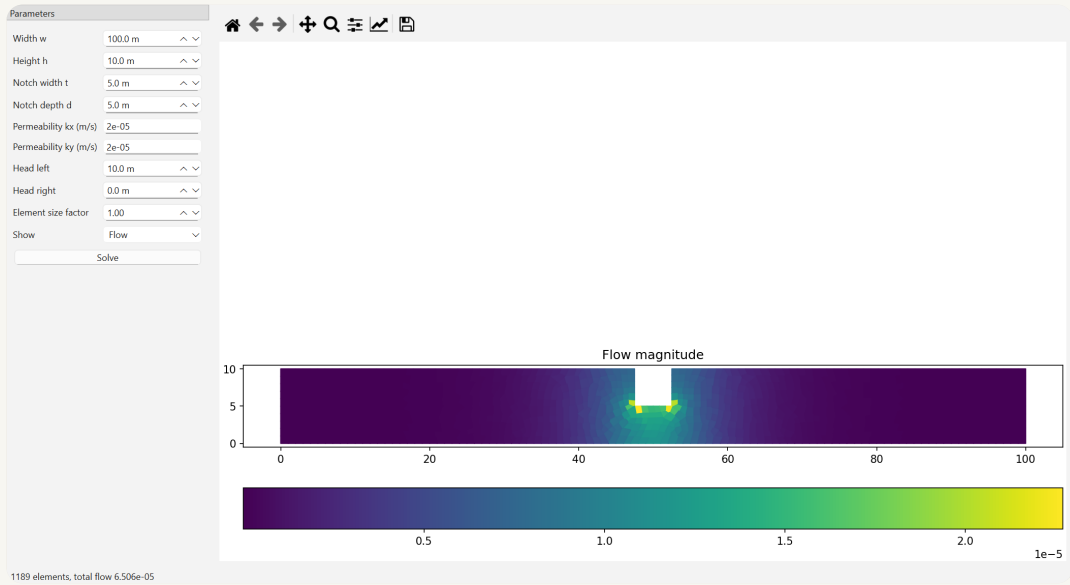
ai/generated/gwflow_ui.py

Plausible in the code, empty on screen

enerated: Show = Flow



Corrected



fv.draw_element_values() has no axes argument. It drew into a hidden pyplot figure, a new one on every redraw, and left the embedded axes mpty. The fix draws a **PolyCollection** on the embedded axes.

Everything that had to be fixed

Problem	Found by	Origin
The flow plot was empty	Running and looking	Generated
A notch deeper or wider than the region hangs Gmsh	Trying invalid input	Generated
Permeability limited to 0.01–100 m/s, 2 decimals	Domain knowledge	Generated
Antisymmetry test tolerance of 1e-6	Running the tests	Generated
One-point integration: rank-deficient elements	Following up the failed test	Original script
"Max flow" grows with mesh refinement	Refining the mesh	Original script

What worked: the model reproduced the original exactly, the worker-thread pattern was correct, and four of five tests were meaningful and passed.

t copied the original's weaknesses too

One-point integration

```
ep = [1.0, 1]
```

Each element matrix has rank 2 instead of 3, which allows spurious hourglass modes. With 2 × 2 integration the antisymmetry error on the finest mesh drops from 0.023 m to 0.004 m.

"Max flow" is not a result

Size factor	1.0	0.5	0.25
Max element flow	0.98	1.13	1.41
Total flow	3.227	3.230	3.230

The notch corners make the exact gradient singular. Total flow converges.

n assistant refactors what you give it. It rarely asks whether the original was right.

Lessons for working with an AI assistant

Run it and look

The empty plot raised no error. Only running the program found it.

Test the physics

Ask for physical checks. Investigate a failure before loosening a tolerance.

Expect inherited errors

A faithful refactoring keeps the original's mistakes.

Validate input

Invalid values reaching a worker thread can hang the whole application.

You own the result

Units, ranges and validity are domain knowledge. Check them yourself.

Use it responsibly

Follow university rules before sharing unpublished code or data. Report AI use in theses and papers.

xercise: run the three prompts on one of your own scripts. Hand in the prompts, the generated code and a review like review.md.

Checklist for your own application

- A window class that builds everything in __init__
- Layouts, not pixel positions
- Spin boxes or validators for numbers
- No slot that blocks: QTimer steps or a worker thread
- A model with no Qt imports, tested on its own
- update_controls() and update_model() between them
- Model/view for large tables
- Files loaded relative to the script, not the working directory

Where to go next

`cases/beam_app_v2`

A complete application: Designer forms, drawing, results window, tests and an installer

`cases/surface · volume · particle`

Main window, docks, custom signals and live 3D with PyVista

`cases/weather_app`

Data from a web API, plotted with matplotlib

`doc.qt.io/qtforpython-6`

Reference, tutorials and many more examples

